

Rapport des Travaux Pratiques

Ingénierie et Maquette Numérique

Réalisé par : AMARA YASSINE

Filière : Ingénierie des systèmes d'information et Big Data (ISIBD)

Niveau : 2^{ème} Année Cycle Ingénieur

Encadré par : Pr. HRIMECH HAMID

Année universitaire : 2025 / 2026

Table des matières

1	TP Git — Branches, Merge et Conflits	3
1.1	Introduction	3
1.2	Initialisation du dépôt et premier commit	3
1.3	Création de la branche <code>dev</code> et commit CSS	4
1.4	Fusion de <code>dev</code> dans <code>master</code> (fast-forward)	5
1.5	Création du fichier <code>readme.md</code> et branches de conflit	6
1.6	Déclenchement et résolution du conflit	6
1.7	Publication et fusion finale dans <code>master</code>	8
2	TP Git — Branche <code>style</code> et Gestion des Conflits	10
2.1	Introduction	10
2.2	Exercice 1 : Branche <code>style</code>	10
2.2.1	Initialisation du projet <code>mon_projet</code>	10
2.2.2	Création de la branche <code>style</code> et modification CSS	10
2.2.3	Modification HTML sur <code>main</code> et isolation des branches	11
2.2.4	Deuxième modification CSS (police) sur <code>style</code>	11
2.3	Exercice 2 : Fusion de <code>style</code> dans <code>main</code>	11
2.4	Exercice 3 : Gestion de conflits avec <code>conflit.txt</code>	12
2.4.1	Initialisation du dépôt <code>tp_conflit</code>	12
2.4.2	Modifications divergentes sur <code>branche1</code> et <code>branche2</code>	13
2.4.3	Déclenchement du conflit	13
2.4.4	Résolution du conflit	14
3	TP Études de Cas — UML et Git : Approche Intégrée	15
3.1	Introduction	15
3.2	Étude de Cas 1 : Plateforme E-commerce	15
3.2.1	Scénario et objectifs	15
3.2.2	Modélisation UML	15
3.2.3	Partie Git — Workflow par branches	16
3.3	Étude de Cas 2 : Application de Gestion Universitaire	18
3.3.1	Scénario et objectifs	18
3.3.2	Modélisation UML	18
3.3.3	Partie Git — Développement par modules	19
3.4	Étude de Cas 3 : Gestion des Conflits Git	21
3.4.1	Scénario et objectifs	21
3.4.2	Simulation et résolution	21
3.5	Étude de Cas 4 : Projet Complet — Système de Réservation d'Hôtel	23
3.5.1	Scénario et objectifs	23
3.5.2	Modélisation UML	23

4 Travaux Pratiques Unity 3D	25
4.1 Introduction	25
4.2 TP 1 — Création d'un Système Solaire Animé	25
4.2.1 Objectif	25
4.2.2 Structure du projet	25
4.2.3 Mise en place de la scène et des textures	26
4.2.4 Animation : rotation et révolution	26
4.2.5 Caméra contrôlée par la souris	27
4.3 TP 2 — Scène 3D Interactive avec Obstacles	27
4.3.1 Objectif	27
4.3.2 Structure du projet	28
4.3.3 Construction de la scène	28
4.4 TP 3 — Mini Application de Réalité Augmentée avec Intelligence Artificielle . .	29
4.4.1 Objectif	29
4.4.2 Technologies utilisées	29
4.4.3 Structure du projet	30
4.4.4 Fonctionnement de l'application	30
4.4.5 Pipeline de développement	31
Conclusion	33

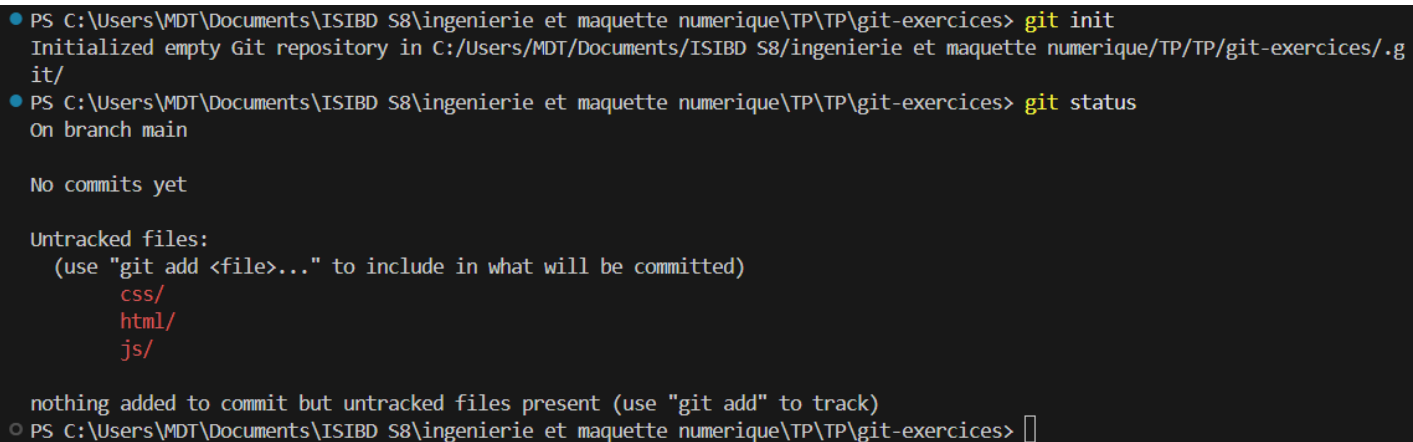
Chapitre 1 TP Git — Branches, Merge et Conflits

1.1 Introduction

Ce premier travail pratique a pour objectif d'apprendre à initialiser un dépôt Git, à organiser un projet en dossiers, à gérer des branches, à fusionner du code et à résoudre des conflits.

1.2 Initialisation du dépôt et premier commit

Le dépôt `git-exercices` est initialisé avec `git init`, puis la structure `css/`, `html/` et `js/` est créée. La commande `git status` confirme que les trois dossiers sont détectés comme fichiers non suivis (*untracked*).



```
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git init
Initialized empty Git repository in C:/Users/MDT/Documents/ISIBD S8/ingenierie et maquette numerique/TP/TP/git-exercices/.git/
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    css/
    html/
    js/

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices>
```

FIGURE 1.1 – `git init` et `git status` : dossiers non suivis détectés

Après `git add *`, les trois fichiers passent en zone de *staging*. Le commit initial est ensuite créé :

```
git add *
git commit -m "Initial commit"
```

```

PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git commit -m "Initial commit"
[main (root-commit) 497aa66] Initial commit
3 files changed, 19 insertions(+)
create mode 100644 css/index.css
create mode 100644 html/index.html
create mode 100644 js/index.js

```

FIGURE 1.2 – Résultat du premier commit : 3 fichiers créés, 19 insertions

Un nouveau `git status` confirme que l'arbre de travail est propre (*nothing to commit, working tree clean*).

```

PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git commit -m "Initial commit"
[main (root-commit) 497aa66] Initial commit
3 files changed, 19 insertions(+)
create mode 100644 css/index.css
create mode 100644 html/index.html
create mode 100644 js/index.js
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git status
On branch main
nothing to commit, working tree clean

```

FIGURE 1.3 – `git status` après le commit : dépôt propre

1.3 Création de la branche dev et commit CSS

Une nouvelle branche `dev` est créée, puis listée avec `git branch`. L'astérisque indique que `master` reste la branche courante.

```

PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git branch dev
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git branch
dev
* master

```

FIGURE 1.4 – Création de la branche `dev` et liste des branches

On bascule sur `dev` avec `git checkout dev`, puis on modifie `style.css` (couleur de fond) et on commite le changement :

```

git checkout dev
git add css/style.css
git commit -m "[CSS] Changed background"

```

```

PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git checkout dev
Switched to branch 'dev'
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git status
On branch dev
nothing to commit, working tree clean

```

FIGURE 1.5 – Basculement sur `dev` et état propre de la branche

```

PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git add css/style.css
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git commit -m "[CSS] Changed backg
round"
[dev b30ced5] [CSS] Changed background
1 file changed, 3 insertions(+)
create mode 100644 css/style.css

```

FIGURE 1.6 – Commit CSS sur la branche dev

La branche dev est ensuite publiée sur GitHub :

```
git push --set-upstream origin dev
```

```

PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git push --set-upstream origin dev

Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 426 bytes | 142.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'dev' on GitHub by visiting:
remote:   https://github.com/Yassine-Amara/git-exercices/pull/new/dev
remote:
To https://github.com/Yassine-Amara/git-exercices.git
 * [new branch]      dev -> dev
branch 'dev' set up to track 'origin/dev'.

```

FIGURE 1.7 – git push -set-upstream origin dev : branche distante créée

1.4 Fusion de dev dans master (fast-forward)

On revient sur master, on effectue un git pull puis un git merge dev. Git effectue un *fast-forward* car master est un ancêtre direct de dev.

```
git checkout master
git pull
git merge dev
```

```

PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git checkout master
D      css/index.css
Switched to branch 'master'
Your branch is up to date with 'origin/master'.
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git pull
Already up to date.
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git merge dev
Updating 497aa66..b30ced5
Fast-forward
 css/style.css | 3 +++
1 file changed, 3 insertions(+)
create mode 100644 css/style.css
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> 

```

FIGURE 1.8 – Fusion dev → master par fast-forward

1.5 Création du fichier `readme.md` et branches de conflit

Un fichier `readme.md` est créé sur `master` et commité. Deux nouvelles branches `branch-a` et `branch-b` sont créées à partir de cet état.

```
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> New-Item readme.md

Répertoire : C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices

Mode                LastWriteTime         Length Name
----                -
-a-----         05/05/2026         18:36             0 readme.md

PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git add readme.md
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git commit -m "Added readme"
[master 1bacf20] Added readme
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 readme.md
```

FIGURE 1.9 – Création de `readme.md` et commit sur `master`

Sur `branch-a`, la ligne *Branch-a instructions* est ajoutée dans `readme.md` et commitée. Sur `branch-b`, la ligne *Branch-b instructions* est ajoutée au même endroit.

```
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git branch branch-a
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git branch branch-b
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git checkout branch-a
D      css/index.css
Switched to branch 'branch-a'
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git add readme.md
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git commit -m "branch-a change"
[branch-a 431a0c7] branch-a change
1 file changed, 1 insertion(+)
```

FIGURE 1.10 – Création de `branch-a` et `branch-b`, commit sur `branch-a`

```
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git checkout branch-b
D      css/index.css
Switched to branch 'branch-b'
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git add readme.md
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git commit -m "branch-b change"
[branch-b 003ef04] branch-b change
1 file changed, 1 insertion(+)
```

FIGURE 1.11 – Commit sur `branch-b` avec une modification divergente

1.6 Déclenchement et résolution du conflit

Une branche `branch-master` est créée à partir de `dev` pour simuler la branche principale. `branch-a` est fusionnée sans problème (*fast-forward*). La fusion de `branch-b` déclenche un conflit car les deux branches ont modifié la même ligne de `readme.md`.

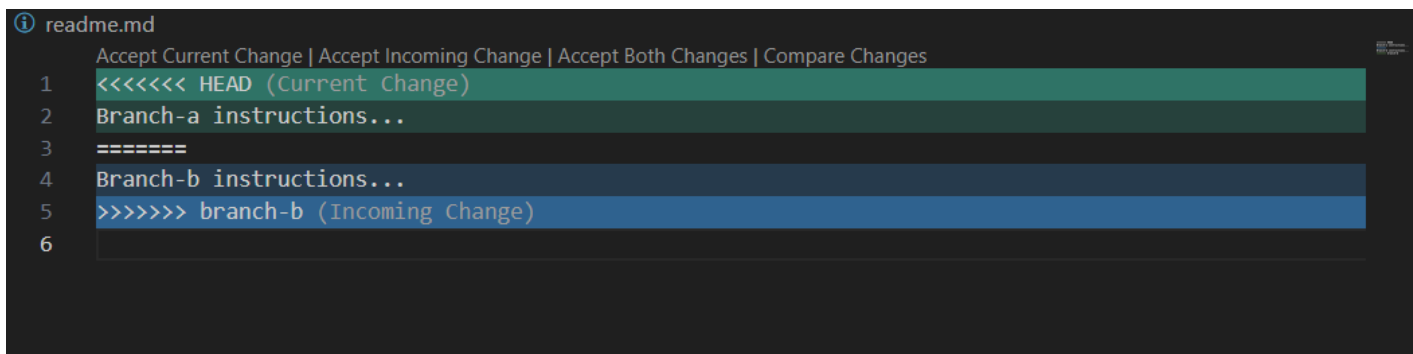
```

PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git checkout dev
D
css/index.css
Switched to branch 'dev'
Your branch is up to date with 'origin/dev'.
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git checkout -b branch-master
Switched to a new branch 'branch-master'
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git merge branch-a
Updating b30ced5..431a0c7
Fast-forward
 readme.md | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 readme.md
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git merge branch-b
Auto-merging readme.md
CONFLICT (content): Merge conflict in readme.md
Automatic merge failed; fix conflicts and then commit the result.

```

FIGURE 1.12 – Conflit détecté lors du merge de branch-b dans branch-master

VS Code affiche les marqueurs de conflit. Les deux versions concurrentes sont clairement identifiées :



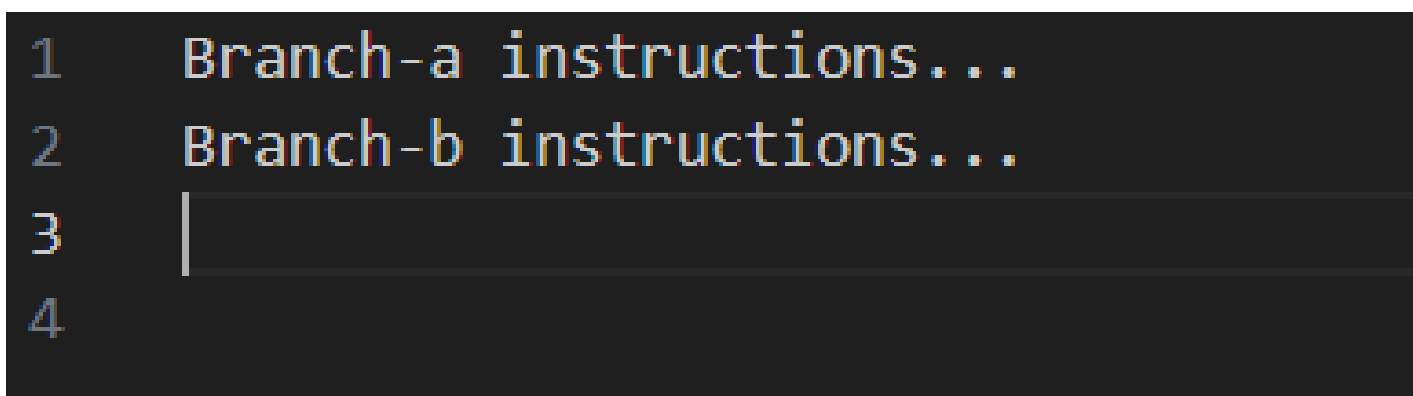
```

i readme.md
Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
1 <<<<<< HEAD (Current Change)
2 Branch-a instructions...
3 =====
4 Branch-b instructions...
5 >>>>>> branch-b (Incoming Change)
6

```

FIGURE 1.13 – Marqueurs de conflit dans readme.md vus dans VS Code

Le conflit est résolu en conservant les deux lignes :



```

1 Branch-a instructions...
2 Branch-b instructions...
3
4

```

FIGURE 1.14 – readme.md après résolution manuelle du conflit

```

git add readme.md
git commit -m "fix conflict"

```



```

PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git add readme.md
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git commit -m "fix conflict"
[branch-master 5a2e128] fix conflict

```

FIGURE 1.15 – Commit de résolution du conflit sur branch-master

1.7 Publication et fusion finale dans master

```
git push --all
```

```

PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git push --all
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 8 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (12/12), 977 bytes | 977.00 KiB/s, done.
Total 12 (delta 5), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (5/5), completed with 1 local object.
To https://github.com/Yassine-Amara/git-exercices.git
  497aa66..1bacf20  master -> master
* [new branch]      branch-a -> branch-a
* [new branch]      branch-b -> branch-b
* [new branch]      branch-master -> branch-master

```

FIGURE 1.16 – git push -all : toutes les branches publiées sur GitHub

```
git checkout master
git merge branch-master
```

```

PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git checkout master
>>
D      css/index.css
Switched to branch 'master'
Your branch is up to date with 'origin/master'.
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git merge branch-master
Updating 1bacf20..5a2e128
Fast-forward
 readme.md | 3 +++
 1 file changed, 3 insertions(+)

```

FIGURE 1.17 – Fusion finale branch-master → master

```
git branch -d dev
git branch -d branch-a
git branch -d branch-b
git branch -d branch-master
```

```
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git branch -d dev
Deleted branch dev (was b30ced5).
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git branch -d branch-a
Deleted branch branch-a (was 431a0c7).
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git branch -d branch-b
Deleted branch branch-b (was 003ef04).
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> git branch -d branch-master
Deleted branch branch-master (was 5a2e128).
PS C:\Users\MDT\Documents\ISIBD S8\ingenierie et maquette numerique\TP\TP\git-exercices> █
```

FIGURE 1.18 – Suppression des branches locales après fusion

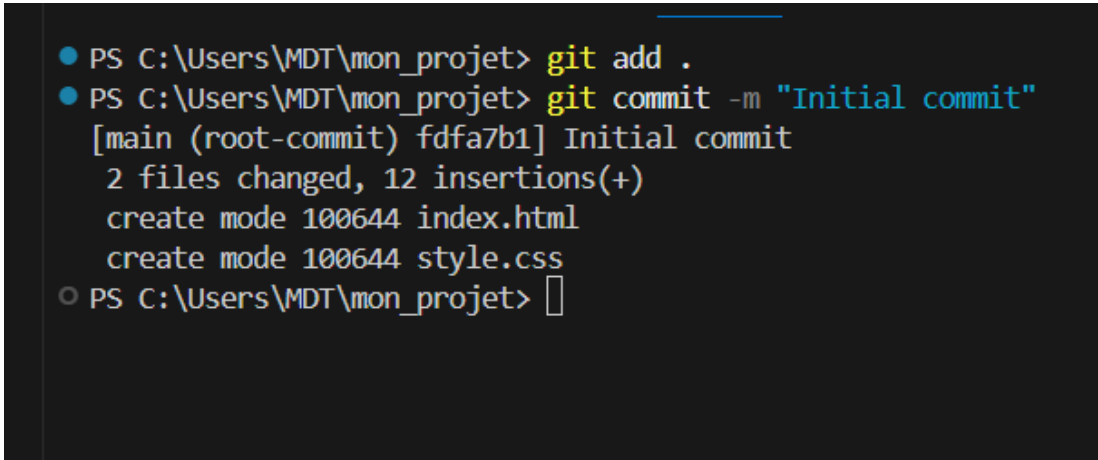
Chapitre 2 TP Git — Branche style et Gestion des Conflits

2.1 Introduction

Ce deuxième TP approfondit la gestion des branches en travaillant sur un projet web minimaliste (`index.html` + `style.css`). Il couvre la fusion avec commit à deux parents, puis la gestion d'un conflit de contenu dans un fichier texte dédié (`conflit.txt`).

2.2 Exercice 1 : Branche style

2.2.1 Initialisation du projet mon_projet



```
● PS C:\Users\MDT\mon_projet> git add .  
● PS C:\Users\MDT\mon_projet> git commit -m "Initial commit"  
[main (root-commit) fdfa7b1] Initial commit  
2 files changed, 12 insertions(+)  
create mode 100644 index.html  
create mode 100644 style.css  
○ PS C:\Users\MDT\mon_projet> █
```

FIGURE 2.1 – Commit initial du projet mon_projet

2.2.2 Création de la branche style et modification CSS

```
git checkout -b style  
git add .  
git commit -m "Changement_couleur_fond"
```

```
● PS C:\Users\MDT\mon_projet> git checkout -b style
  Switched to a new branch 'style'
● PS C:\Users\MDT\mon_projet> git add .
● PS C:\Users\MDT\mon_projet> git commit -m "Changement couleur fond"
  [style 6bb2092] Changement couleur fond
    1 file changed, 1 insertion(+), 1 deletion(-)
○ PS C:\Users\MDT\mon_projet> █
```

FIGURE 2.2 – Création de la branche `style` et commit du changement de couleur

2.2.3 Modification HTML sur main et isolation des branches

```
git checkout main
git add .
git commit -m "Modification HTML"
```

```
● PS C:\Users\MDT\mon_projet> git checkout main
  Switched to branch 'main'
● PS C:\Users\MDT\mon_projet> git add .
● PS C:\Users\MDT\mon_projet> git commit -m "Modification HTML"
  [main 2e64a76] Modification HTML
    1 file changed, 1 insertion(+), 1 deletion(-)
```

FIGURE 2.3 – Commit de modification HTML sur la branche `main`

2.2.4 Deuxième modification CSS (police) sur style

```
git checkout style
git add .
git commit -m "Changement police"
```

```
● PS C:\Users\MDT\mon_projet> git checkout style
  Switched to branch 'style'
● PS C:\Users\MDT\mon_projet> git add .
● PS C:\Users\MDT\mon_projet> git commit -m "Changement police"
  [style ce9ef91] Changement police
    1 file changed, 1 insertion(+), 1 deletion(-)
○ PS C:\Users\MDT\mon_projet> █
```

FIGURE 2.4 – Second commit CSS (changement de police) sur la branche `style`

2.3 Exercice 2 : Fusion de style dans main

```
git checkout main
git merge style
git log --oneline --graph
```

```

PS C:\Users\MDT\mon_projet> git checkout main
Switched to branch 'main'
PS C:\Users\MDT\mon_projet> git merge style
Merge made by the 'ort' strategy.
 style.css | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\MDT\mon_projet> git log --oneline --graph
*   a396c8f (HEAD -> main) Merge branch 'style'
| \
| * ce9ef91 (style) Changement police
| * 6bb2092 Changement couleur fond
| * | 2e64a76 Modification HTML
| /
* fdfa7b1 Initial commit
PS C:\Users\MDT\mon_projet>

```

FIGURE 2.5 – Fusion style → main et historique graphique montrant le commit à deux parents

2.4 Exercice 3 : Gestion de conflits avec conflit.txt

2.4.1 Initialisation du dépôt tp_conflict

```

git add .
git commit -m "Ajout_fichier_conflict"

```

```

● PS C:\Users\MDT\tp_conflict> git add .
● PS C:\Users\MDT\tp_conflict> git commit -m "Ajout fichier conflit"
[main (root-commit) 10e9248] Ajout fichier conflit
 1 file changed, 3 insertions(+)
 create mode 100644 conflit.txt
○ PS C:\Users\MDT\tp_conflict>

```

FIGURE 2.6 – Commit initial du fichier conflit.txt

2.4.2 Modifications divergentes sur branche1 et branche2

```
PS C:\Users\MDT\tp_conflit> git checkout -b branche1
Switched to a new branch 'branche1'
PS C:\Users\MDT\tp_conflit> git commit -am "modif branche1"
[branche1 d6269b7] modif branche1
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\MDT\tp_conflit>
```

FIGURE 2.7 – Création de branche1 et modification de conflit.txt

```
PS C:\Users\MDT\tp_conflit> git checkout main
Switched to branch 'main'
PS C:\Users\MDT\tp_conflit> git checkout -b branche2
Switched to a new branch 'branche2'
PS C:\Users\MDT\tp_conflit> git commit -am "modif branche2"
[branche2 9e38272] modif branche2
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\MDT\tp_conflit>
```

FIGURE 2.8 – Création de branche2 et modification divergente de conflit.txt

2.4.3 Déclenchement du conflit

```
git checkout branche1
git merge branche2
```

```
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\MDT\tp_conflit> git checkout branche1
Switched to branch 'branche1'
PS C:\Users\MDT\tp_conflit> git merge branche2
Auto-merging conflit.txt
CONFLICT (content): Merge conflict in conflit.txt
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\MDT\tp_conflit>
```

FIGURE 2.9 – Conflit détecté lors du merge de branche2 dans branche1

2.4.4 Résolution du conflit

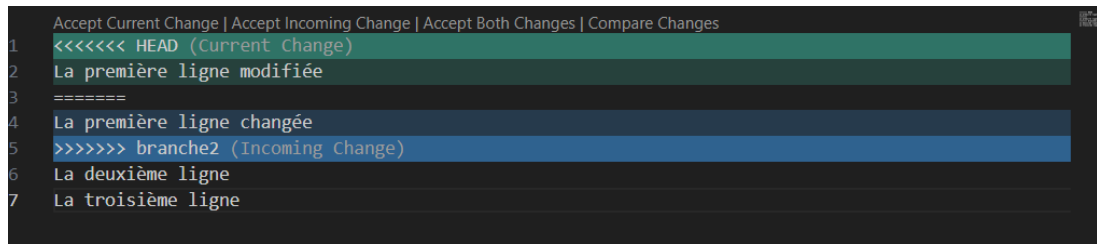


FIGURE 2.10 – Marqueurs de conflit dans `conflit.txt` affichés par VS Code

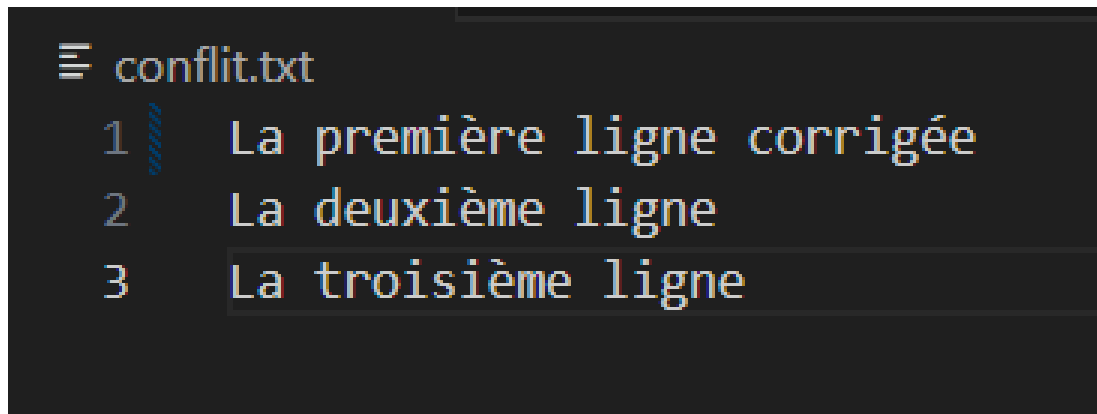


FIGURE 2.11 – `conflit.txt` après résolution manuelle du conflit

```
git add conflit.txt
git commit -m "Résolution du conflit"
git log --oneline --graph
```

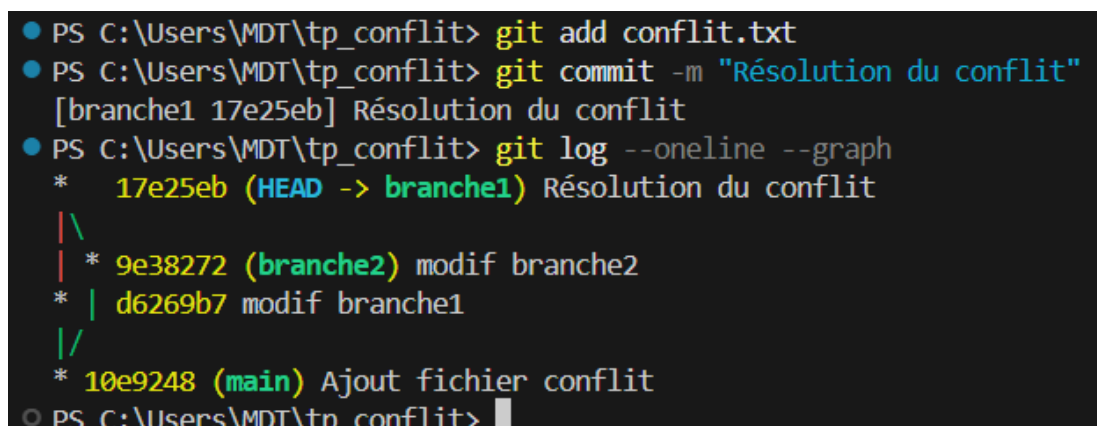


FIGURE 2.12 – Historique après résolution : commit de fusion à deux parents

Chapitre 3 TP Études de Cas — UML et Git : Approche Intégrée

3.1 Introduction

Ce troisième TP combine la modélisation UML avec la gestion de versions Git à travers quatre études de cas progressives. L'objectif est de relier la conception logicielle (diagrammes UML) à l'implémentation et au travail collaboratif (branches Git), en adoptant un workflow professionnel.

3.2 Étude de Cas 1 : Plateforme E-commerce

3.2.1 Scénario et objectifs

Une équipe développe une plateforme e-commerce permettant la consultation des produits, l'ajout au panier et le paiement en ligne. L'objectif est de modéliser ce système en UML puis d'implémenter les fonctionnalités via des branches Git dédiées.

3.2.2 Modélisation UML

Diagramme de cas d'utilisation

Le système implique deux acteurs principaux : le **Client** et l'**Administrateur**. Le client peut consulter les produits, ajouter au panier et payer. L'administrateur gère le catalogue de produits.

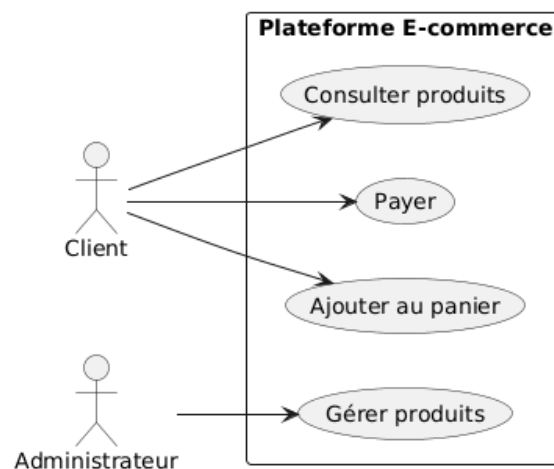


FIGURE 3.1 – Diagramme de cas d'utilisation — Plateforme E-commerce

Diagramme de classes

Le modèle objet de la plateforme comprend les classes **Client**, **Panier**, **Produit**, **Commande** et **Paieement**, reliées par des associations avec multiplicités.

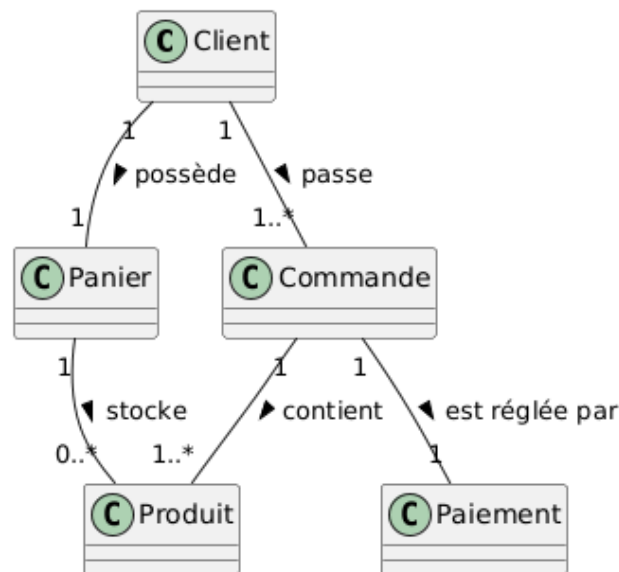


FIGURE 3.2 – Diagramme de classes — Plateforme E-commerce

Diagramme de séquence

Le scénario de paiement implique trois participants : le **Client**, le **Système** et le **ServicePaiement**. Le client déclenche le paiement, le système délègue la vérification au service de paiement qui renvoie une confirmation.

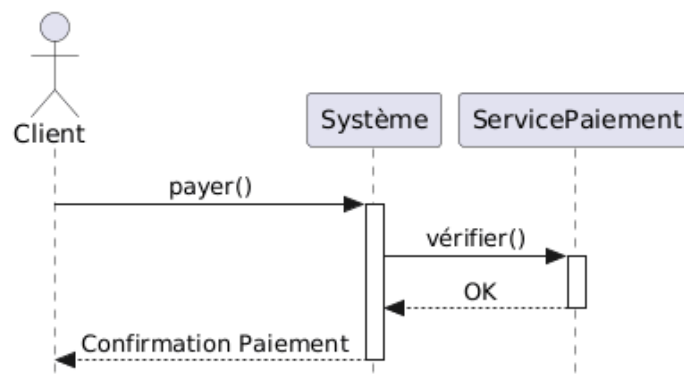


FIGURE 3.3 – Diagramme de séquence — Scénario de paiement

3.2.3 Partie Git — Workflow par branches

La stratégie de branches adoptée suit le modèle *Git Flow* : **main** pour la production, **develop** pour l'intégration, et des branches **feature/*** pour chaque fonctionnalité.

Deux branches de fonctionnalité sont créées en parallèle : **feature/catalogue** (classes **Produit** et **Client**) et **feature/paiement** (classes **Paieement** et **Commande**). Chacune est fusionnée dans **develop** avec l'option **-no-ff** pour préserver l'historique des branches.

```

git checkout -b feature/catalogue
git commit -m "feat: ajout classes Produit et Client"
git checkout develop
git merge --no-ff feature/catalogue -m "merge: intégration catalogue"
git checkout -b feature/paiement
git commit -m "feat: ajout module paiement en ligne"
git checkout develop
git merge --no-ff feature/paiement -m "merge: intégration paiement"

```

```

• PS C:\Users\MDT\tp-uml-git> cd ecommerce
• PS C:\Users\MDT\tp-uml-git\ecommerce> git init
  Initialized empty Git repository in C:/Users/MDT/tp-uml-git/ecommerce/.git/
• PS C:\Users\MDT\tp-uml-git\ecommerce> echo "# Plateforme E-commerce" > README.md
• PS C:\Users\MDT\tp-uml-git\ecommerce> git add README.md
• PS C:\Users\MDT\tp-uml-git\ecommerce> git commit -m "init: création du projet e-commerce"
>>
[main (root-commit) e28e046] init: création du projet e-commerce
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 README.md
• PS C:\Users\MDT\tp-uml-git\ecommerce> git checkout -b develop
  Switched to a new branch 'develop'
• PS C:\Users\MDT\tp-uml-git\ecommerce> git checkout -b feature/catalogue
  Switched to a new branch 'feature/catalogue'
• PS C:\Users\MDT\tp-uml-git\ecommerce> echo "class Produit {}" > Produit.java
• PS C:\Users\MDT\tp-uml-git\ecommerce> echo "class Client {}" > Client.java
• PS C:\Users\MDT\tp-uml-git\ecommerce> git add .
• PS C:\Users\MDT\tp-uml-git\ecommerce> git commit -m "feat: ajout classes Produit et Client"
[feature/catalogue 667b899] feat: ajout classes Produit et Client
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 Client.java
 create mode 100644 Produit.java
• PS C:\Users\MDT\tp-uml-git\ecommerce> git checkout develop
  Switched to branch 'develop'
• PS C:\Users\MDT\tp-uml-git\ecommerce> git checkout -b feature/paiement
  Switched to a new branch 'feature/paiement'
• PS C:\Users\MDT\tp-uml-git\ecommerce> echo "class Paiement {}" > Paiement.java
• PS C:\Users\MDT\tp-uml-git\ecommerce> echo "class Commande {}" > Commande.java
• PS C:\Users\MDT\tp-uml-git\ecommerce> git add .
• PS C:\Users\MDT\tp-uml-git\ecommerce> git commit -m "feat: ajout module paiement en ligne"
[feature/paiement 3828453] feat: ajout module paiement en ligne
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 Commande.java
 create mode 100644 Paiement.java
• PS C:\Users\MDT\tp-uml-git\ecommerce> git checkout develop
  Switched to branch 'develop'
• PS C:\Users\MDT\tp-uml-git\ecommerce> git merge --no-ff feature/catalogue -m "merge: intégration catalogue"
Merge made by the 'ort' strategy.
 Client.java | Bin 0 -> 36 bytes
 Produit.java | Bin 0 -> 38 bytes
 2 files changed, 0 insertions(+), 0 deletions(-)

```

FIGURE 3.4 – Initialisation du projet e-commerce, création des branches feature et commits

La commande `git log -oneline -graph -all` révèle l'historique complet avec les deux branches fusionnées dans develop :

```

● PS C:\Users\MDT\tp-uml-git\ecommerce> git merge --no-ff feature/catalogue -m "merge: intégration catalogue"
Merge made by the 'ort' strategy.
 Client.java | Bin 0 -> 36 bytes
 Produit.java | Bin 0 -> 38 bytes
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 Client.java
 create mode 100644 Produit.java
● PS C:\Users\MDT\tp-uml-git\ecommerce> git merge --no-ff feature/paiement -m "merge: intégration paiement"
Merge made by the 'ort' strategy.
 Commande.java | Bin 0 -> 40 bytes
 Paiement.java | Bin 0 -> 40 bytes
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 Commande.java
 create mode 100644 Paiement.java
● PS C:\Users\MDT\tp-uml-git\ecommerce> git log --oneline --graph --all
* 074bccf (HEAD -> develop) merge: intégration paiement
| \
| * 3828453 (feature/paiement) feat: ajout module paiement en ligne
| * | 7e5e8a8 merge: intégration catalogue
| \ \
| | /
| / |
| * 667b899 (feature/catalogue) feat: ajout classes Produit et Client
| /
| * e28e046 (main) init: création du projet e-commerce
○ PS C:\Users\MDT\tp-uml-git\ecommerce>

```

FIGURE 3.5 – Historique graphique après fusion des deux branches feature dans develop

3.3 Étude de Cas 2 : Application de Gestion Universitaire

3.3.1 Scénario et objectifs

Ce cas modélise une application de gestion universitaire gérant les étudiants, les enseignants et les cours. Il met en pratique l'héritage UML et un workflow Git collaboratif avec plusieurs branches de fonctionnalité.

3.3.2 Modélisation UML

Diagramme de classes

Le modèle utilise une superclasse `Personne` dont héritent `Étudiant` et `Enseignant`. La classe `Cours` est associée aux deux sous-classes via des relations de multiplicité.

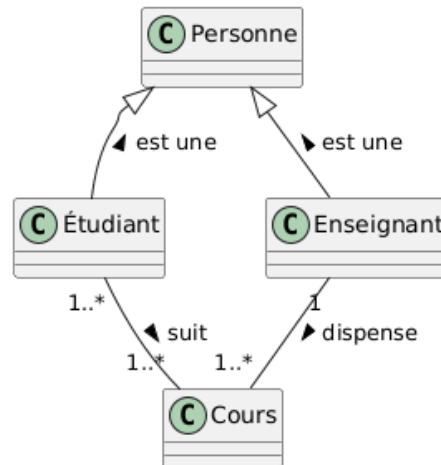
Diagramme de Classes - Gestion Universitaire

FIGURE 3.6 – Diagramme de classes — Gestion Universitaire (héritage)

Diagramme de séquence

Le scénario d'inscription d'un étudiant met en jeu trois participants : l'**Étudiant**, le **Système** et la base de données **BD**. L'étudiant envoie une demande d'inscription, le système enregistre les données et retourne une confirmation.

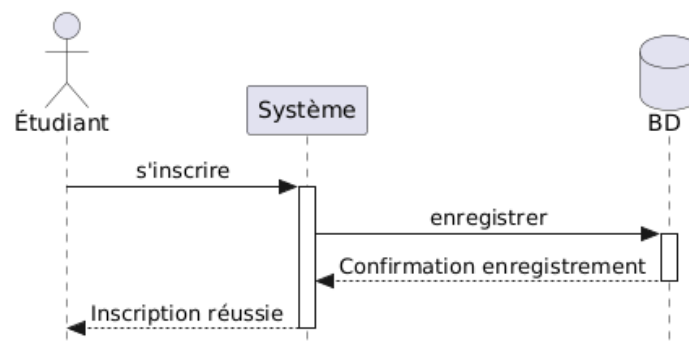


FIGURE 3.7 – Diagramme de séquence — Inscription d'un étudiant

3.3.3 Partie Git — Développement par modules

Trois branches de fonctionnalité sont créées, une par module métier : `feature/gestion-etudiants`, `feature/gestion-cours` et `feature/gestion-enseignants`. Chacune est fusionnée successivement dans `develop` avec `-no-ff`.

```

• PS C:\Users\MDT\tp-uml-git> cd university-app
• PS C:\Users\MDT\tp-uml-git\university-app> git init
Initialized empty Git repository in C:/Users/MDT/tp-uml-git/university-app/.git/
• PS C:\Users\MDT\tp-uml-git\university-app> echo "# Gestion Universitaire" > README.md
PS C:\Users\MDT\tp-uml-git\university-app> git add . && git commit -m "init: projet gestion universitaire"
Au caractère Ligne:1 : 11
+ git add . && git commit -m "init: projet gestion universitaire"
+
+ ~~~
Le jeton « && » n'est pas un séparateur d'instruction valide.
+ CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordException
+ FullyQualifiedErrorId : InvalidEndOfLine

• PS C:\Users\MDT\tp-uml-git\university-app> git add .
• PS C:\Users\MDT\tp-uml-git\university-app> git commit -m "init: projet gestion universitaire"
[main (root-commit) 42638d9] init: projet gestion universitaire
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 README.md
• PS C:\Users\MDT\tp-uml-git\university-app> git checkout -b develop
Switched to a new branch 'develop'
• PS C:\Users\MDT\tp-uml-git\university-app> git checkout -b feature/gestion-etudiants
Switched to a new branch 'feature/gestion-etudiants'
• PS C:\Users\MDT\tp-uml-git\university-app> echo "class Etudiant extends Personne {}" > Etudiant.java
• PS C:\Users\MDT\tp-uml-git\university-app> git add .
• PS C:\Users\MDT\tp-uml-git\university-app> git commit -m "feat: classe Etudiant avec héritage Personne"
[feature/gestion-etudiants 83fa759] feat: classe Etudiant avec héritage Personne
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 Etudiant.java
• PS C:\Users\MDT\tp-uml-git\university-app> echo "void sInscrire() {}" >> Etudiant.java
• PS C:\Users\MDT\tp-uml-git\university-app> git add .
• PS C:\Users\MDT\tp-uml-git\university-app> git commit -m "feat: méthode inscription étudiant"
[feature/gestion-etudiants 37df799] feat: méthode inscription étudiant
1 file changed, 0 insertions(+), 0 deletions(-)
• PS C:\Users\MDT\tp-uml-git\university-app> git checkout develop
Switched to branch 'develop'
• PS C:\Users\MDT\tp-uml-git\university-app> git checkout -b feature/gestion-cours
Switched to a new branch 'feature/gestion-cours'
• PS C:\Users\MDT\tp-uml-git\university-app> echo "class Cours {}" > Cours.java
• PS C:\Users\MDT\tp-uml-git\university-app> git add .
• PS C:\Users\MDT\tp-uml-git\university-app> git commit -m "feat: classe Cours avec crédits et places"

```

FIGURE 3.8 – Initialisation de university-app, branches feature et commits par module

```

    create mode 100644 Enseignant.java
PS C:\Users\MDT\tp-uml-git\university-app> git checkout develop
PS C:\Users\MDT\tp-uml-git\university-app> git merge --no-ff feature/gestion-etudiants -m "merge: module étudiants"
Merge made by the 'ort' strategy.
Etudiant.java | Bin 0 -> 116 bytes
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 Etudiant.java
PS C:\Users\MDT\tp-uml-git\university-app> git merge --no-ff feature/gestion-cours -m "merge: module cours"
Merge made by the 'ort' strategy.
Cours.java | Bin 0 -> 34 bytes
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 Cours.java
PS C:\Users\MDT\tp-uml-git\university-app> git merge --no-ff feature/gestion-enseignants -m "merge: module enseignants"
Merge made by the 'ort' strategy.
Enseignant.java | Bin 0 -> 78 bytes
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 Enseignant.java
PS C:\Users\MDT\tp-uml-git\university-app> git log --oneline --graph --all
*   bdb4c71 (HEAD -> develop) merge: module enseignants
| \
| * 6bc8498 (feature/gestion-enseignants) feat: classe Enseignant avec spécialité
* | f9c054f merge: module cours
| \ \
| * | c9dd08b (feature/gestion-cours) feat: classe Cours avec crédits et places
| | /
* | | c3ed24a merge: module étudiants
| \ \
| | /
| | /
| * 37df799 (feature/gestion-etudiants) feat: méthode inscription étudiant
| * 83fa759 feat: classe Etudiant avec héritage Personne
| /
* 42638d9 (main) init: projet gestion universitaire
PS C:\Users\MDT\tp-uml-git\university-app> git branch -a
* develop
  feature/gestion-cours
  feature/gestion-enseignants
  feature/gestion-etudiants
  main

```

FIGURE 3.9 – Fusion des trois modules dans develop et historique graphique complet

3.4 Étude de Cas 3 : Gestion des Conflits Git

3.4.1 Scénario et objectifs

Ce cas simule une situation réelle de conflit : deux développeurs modifient simultanément le même fichier de configuration (`config.txt`). L'objectif est de comprendre l'origine des conflits et de maîtriser leur résolution.

3.4.2 Simulation et résolution

Un dépôt `git-conflicts` est créé avec un fichier `config.txt` initial. Deux branches `dev1` et `dev2` modifient la même variable (`DATABASE_HOST`) avec des valeurs différentes : `localhost` pour `dev1` et `192.168.1.10` pour `dev2`. La fusion de `dev1` dans `main` se déroule sans problème (fast-forward). La fusion de `dev2` déclenche un conflit.

```

git init
git commit -m "init: fichier de configuration"

```

```

git checkout -b dev1
echo 'DATABASE_HOST= "localhost"' > config.txt
git commit -m "config: dev1 utilise base locale"
git checkout main
git checkout -b dev2
echo 'DATABASE_HOST= "192.168.1.10"' > config.txt
git commit -m "config: dev2 utilise serveur r seau"
git checkout main
git merge dev1      % Fast-forward OK
git merge dev2      % CONFLIT

```

```

• PS C:\Users\MDT\tp-uml-git> cd git-conflicts
• PS C:\Users\MDT\tp-uml-git\git-conflicts> git init
  Initialized empty Git repository in C:/Users/MDT/tp-uml-git/git-conflicts/.git/
• PS C:\Users\MDT\tp-uml-git\git-conflicts> echo 'DATABASE_HOST = "production.server.com"' > config.txt
• PS C:\Users\MDT\tp-uml-git\git-conflicts> git add config.txt
• PS C:\Users\MDT\tp-uml-git\git-conflicts> git commit -m "init: fichier de configuration"
[main (root-commit) ee0347d] init: fichier de configuration
  1 file changed, 0 insertions(+), 0 deletions(-)
  create mode 100644 config.txt
• PS C:\Users\MDT\tp-uml-git\git-conflicts> git checkout -b dev1
  Switched to a new branch 'dev1'
• PS C:\Users\MDT\tp-uml-git\git-conflicts> echo 'DATABASE_HOST = "localhost"' > config.txt
• PS C:\Users\MDT\tp-uml-git\git-conflicts> git add config.txt
• PS C:\Users\MDT\tp-uml-git\git-conflicts> git commit -m "config: dev1 utilise base locale"
[dev1 e1e824a] config: dev1 utilise base locale
  1 file changed, 0 insertions(+), 0 deletions(-)
• PS C:\Users\MDT\tp-uml-git\git-conflicts> git checkout main
  Switched to branch 'main'
• PS C:\Users\MDT\tp-uml-git\git-conflicts> git checkout -b dev2
  Switched to a new branch 'dev2'
• PS C:\Users\MDT\tp-uml-git\git-conflicts> echo 'DATABASE_HOST = "192.168.1.10"' > config.txt
• PS C:\Users\MDT\tp-uml-git\git-conflicts> git add config.txt
• PS C:\Users\MDT\tp-uml-git\git-conflicts> git commit -m "config: dev2 utilise serveur réseau"
[dev2 0e6ba37] config: dev2 utilise serveur réseau
  1 file changed, 0 insertions(+), 0 deletions(-)
• PS C:\Users\MDT\tp-uml-git\git-conflicts> git checkout main
  Switched to branch 'main'
• PS C:\Users\MDT\tp-uml-git\git-conflicts> git merge dev1
  Updating ee0347d..e1e824a
  Fast-forward
  config.txt | Bin 84 -> 60 bytes
  1 file changed, 0 insertions(+), 0 deletions(-)

```

FIGURE 3.10 – Simulation du conflit sur config.txt entre dev1 et dev2

3.5 Étude de Cas 4 : Projet Complet — Système de Réserve- ration d'Hôtel

3.5.1 Scénario et objectifs

Cette étude de cas intègre l'ensemble des compétences acquises : modélisation UML complète et workflow Git professionnel appliqués à un système de réservation d'hôtel. Le système permet à un client de réserver une chambre, de procéder au paiement et d'annuler une réservation.

3.5.2 Modélisation UML

Diagramme de cas d'utilisation

L'acteur principal est le **Client**. Les cas d'utilisation identifiés sont : *Réserver chambre*, *Payer* et *Annuler*.

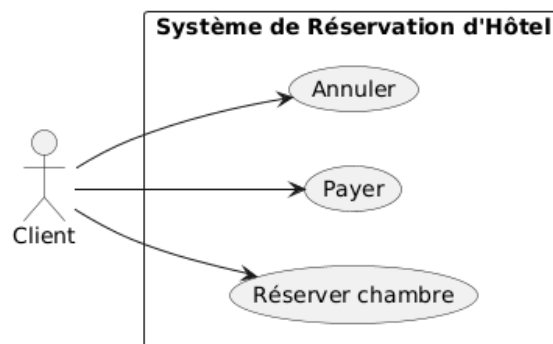


FIGURE 3.11 – Diagramme de cas d'utilisation — Système de Réservation d'Hôtel

Diagramme de classes

Le modèle comprend les classes **Client**, **Réservation**, **Chambre** et **Paie-ment**. La **Réservation** est au cœur du système : elle est effectuée par un **Client**, concerne une **Chambre** et génère un **Paie-ment**.

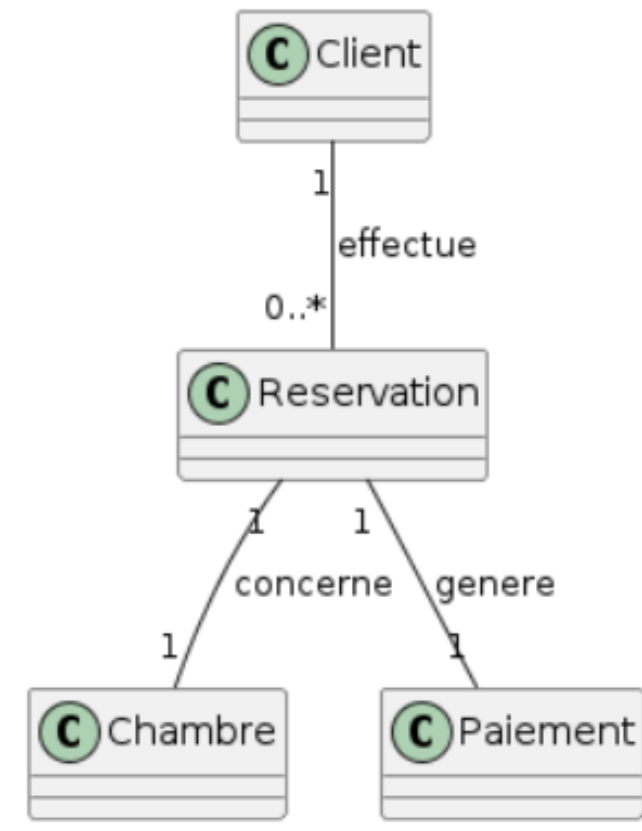


FIGURE 3.12 – Diagramme de classes — Système de Réservation d’Hôtel

Diagramme de séquence

Le scénario de réservation implique quatre participants : le **Client**, le **Système**, la base de données **BD** et le service **Paiement**. Le système vérifie la disponibilité de la chambre auprès de la BD, puis délègue la validation du paiement, avant de retourner une confirmation au client.

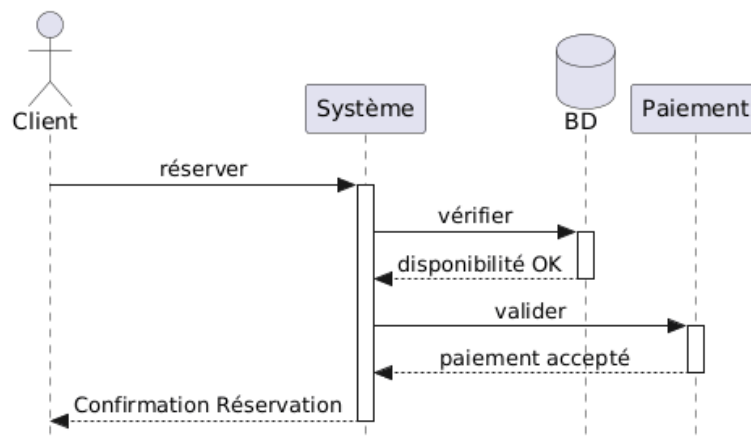


FIGURE 3.13 – Diagramme de séquence — Scénario de réservation avec paiement

Chapitre 4 Travaux Pratiques Unity 3D

4.1 Introduction

Ce chapitre regroupe trois travaux pratiques réalisés sous Unity 3D, couvrant progressivement la création d’une scène 3D interactive, la modélisation d’un système solaire animé, et le développement d’une application de réalité augmentée intégrant de l’intelligence artificielle.

4.2 TP 1 — Création d’un Système Solaire Animé

4.2.1 Objectif

L’objectif de ce TP est de créer une scène animée sous Unity 3D représentant un système solaire. Les compétences visées sont la maîtrise des rotations et translations d’objets, l’utilisation des Skybox et l’application de textures sur des sphères.

4.2.2 Structure du projet

Le projet Unity est organisé selon une arborescence claire dans le panneau Assets. Les dossiers principaux sont les suivants : `Scenes`, `Settings`, `TutorialInfo`, ainsi que des scripts C# tels que `ClickableObject`, `DataLogger`, `IAFollower`, `PlayerMovement` et `log_user`. Le package `InputSystem` est également présent pour la gestion des entrées.

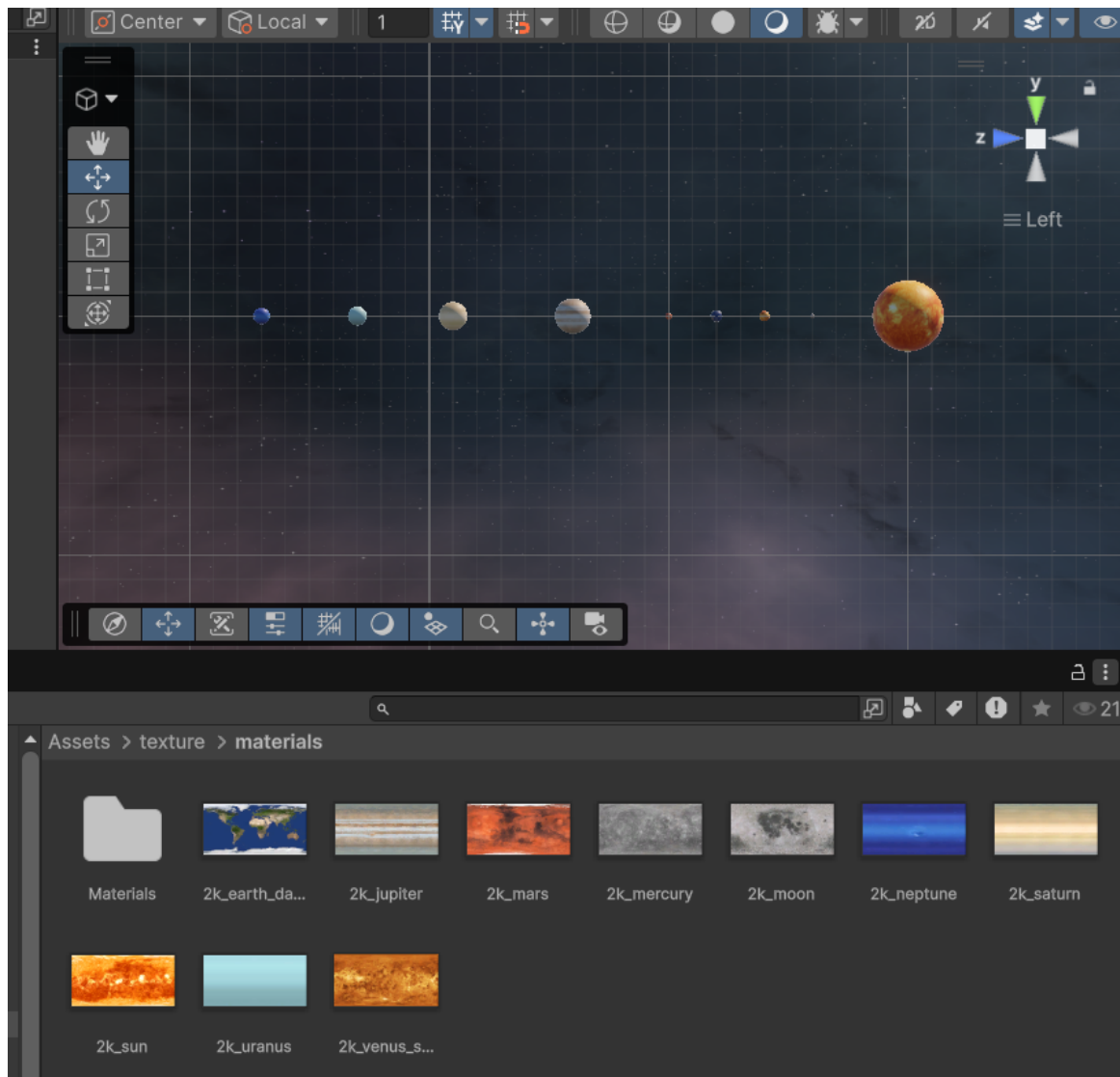


FIGURE 4.1 – Structure du projet Unity — panneau Assets avec scripts et dossiers

4.2.3 Mise en place de la scène et des textures

Les textures des planètes et du soleil sont importées depuis la bibliothèque Solar System Scope. Un dossier **Textures/Materials** est créé dans Assets pour organiser les matériaux. Chaque sphère (Soleil, Mercure, Vénus, Terre, Mars, Jupiter, Saturne, Uranus, Neptune) reçoit ensuite sa texture correspondante via un matériau dédié.

La Skybox *Starfield* est configurée dans les paramètres de rendu de la scène afin de simuler l'environnement spatial. La vue de la scène montre les planètes alignées sur l'axe horizontal avec le Soleil positionné à l'origine.

Les textures disponibles dans le dossier **Assets/texture/materials** sont : `2k_earth_daymap`, `2k_jupiter`, `2k_mars`, `2k_mercury`, `2k_moon`, `2k_neptune`, `2k_saturn`, `2k_sun`, `2k_uranus` et `2k_venus_surface`.

4.2.4 Animation : rotation et révolution

Chaque planète est animée par un script C# qui combine deux mouvements : une rotation sur elle-même et une révolution autour du Soleil. Le script utilise `transform.Rotate` pour la

rotation propre et `transform.RotateAround` pour l'orbite.

```
// Rotation de la planete sur elle-meme
transform.Rotate(0, 35 * Time.deltaTime, 0);

// Revolution autour du Soleil
GameObject Soleil = GameObject.Find("Soleil");
transform.RotateAround(
    Soleil.transform.position,
    -Soleil.transform.up,
    70 * Time.deltaTime
);
```

4.2.5 Caméra contrôlée par la souris

Une caméra de suivi est configurée à l'aide du script `MouseAimCamera`. La caméra suit une cible (`target`) et pivote horizontalement en fonction du mouvement de la souris sur l'axe X. L'offset initial est calculé dans la méthode `Start()` et la rotation est appliquée dans `LateUpdate()`.

```
public class MouseAimCamera : MonoBehaviour {
    public GameObject target;
    public float rotateSpeed = 5;
    Vector3 offset;

    void Start() {
        offset = target.transform.position - transform.position;
    }

    void LateUpdate() {
        float horizontal = Input.GetAxis("Mouse_X") * rotateSpeed;
        target.transform.Rotate(0, horizontal, 0);

        float desiredAngle = target.transform.eulerAngles.y;
        Quaternion rotation = Quaternion.Euler(0, desiredAngle, 0);
        transform.position = target.transform.position - (rotation *
            offset);

        transform.LookAt(target.transform);
    }
}
```

4.3 TP 2 — Scène 3D Interactive avec Obstacles

4.3.1 Objectif

Ce TP consiste à créer une scène 3D Unity comportant un sol plan et plusieurs objets primitifs servant d'obstacles. Il introduit la manipulation d'objets dans l'espace 3D, la gestion des collisions et la navigation d'un personnage dans un environnement structuré.

4.3.2 Structure du projet

Le projet utilise le pipeline de rendu URP (Universal Render Pipeline). La structure Assets comprend les dossiers `Scenes`, `Settings` et `TutorialInfo`, ainsi que les scripts `ClickableObject`, `DataLogger`, `IAFollower` et `PlayerMovement`.

4.3.3 Construction de la scène

La scène comporte un plan horizontal servant de sol et plusieurs objets primitifs disposés comme obstacles : des cubes, des cylindres et des sphères de tailles variées. Ces objets sont positionnés manuellement dans la scène à l'aide des outils de transformation Unity.

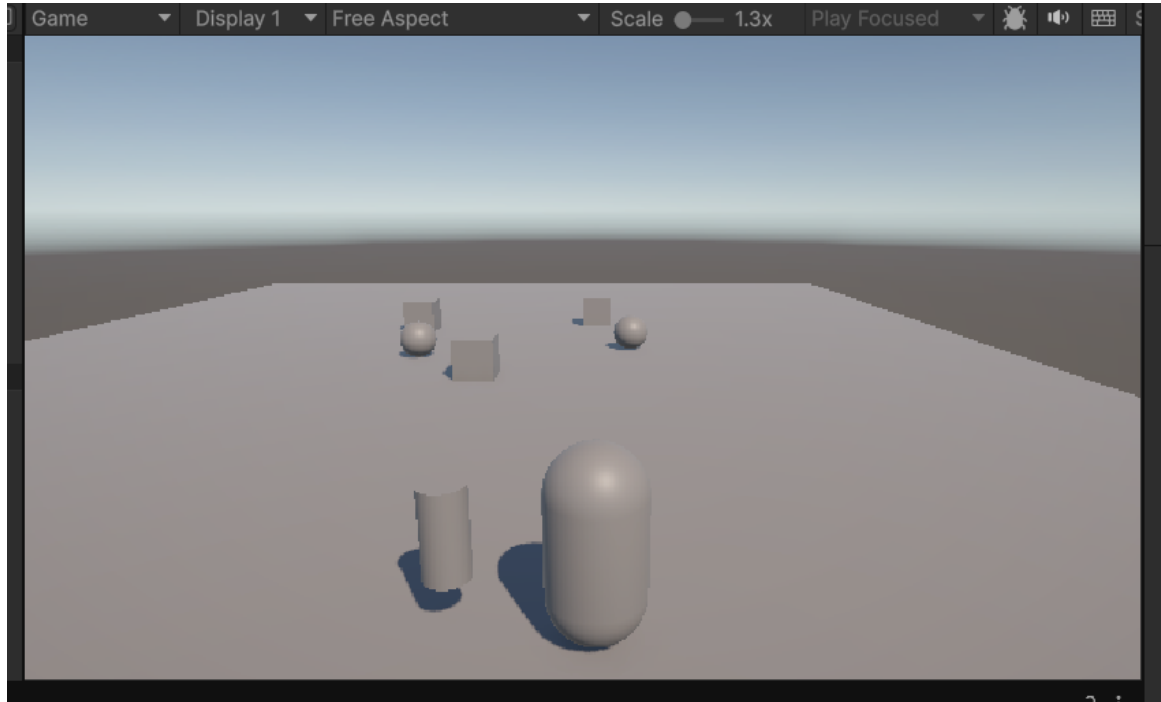


FIGURE 4.2 – Vue de la scène Unity — sol et obstacles primitifs (vue rapprochée)

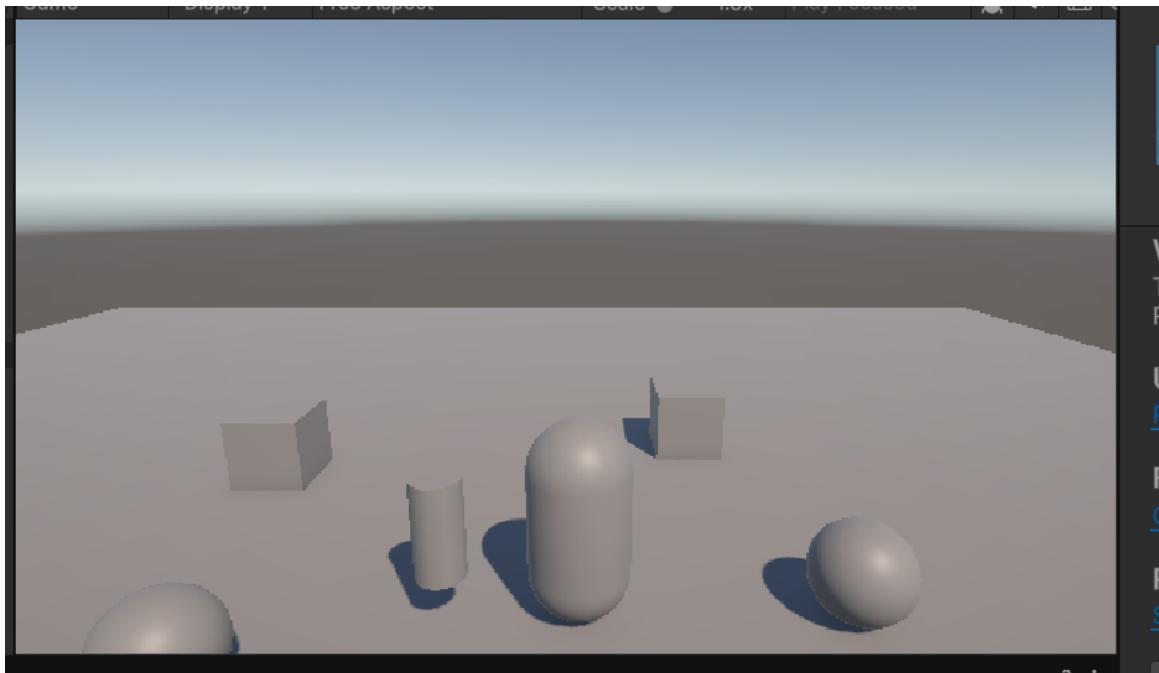


FIGURE 4.3 – Vue de la scène Unity — disposition générale des obstacles depuis la caméra de jeu

La scène est visualisée depuis la caméra de jeu (onglet *Game*) en mode *Play Focused* avec un rendu URP, confirmé par l’affichage du panneau de bienvenue SRP dans l’Inspecteur.

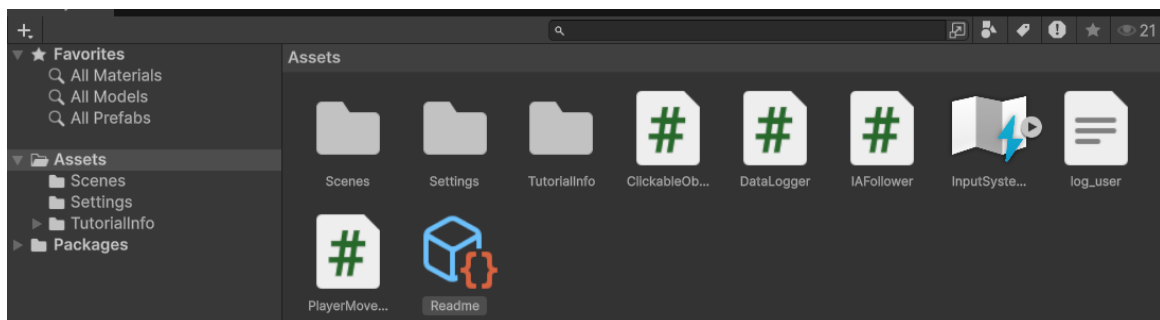


FIGURE 4.4 – Vue de la scène Unity — panneau Assets avec scripts du projet obstacles

4.4 TP 3 — Mini Application de Réalité Augmentée avec Intelligence Artificielle

4.4.1 Objectif

Ce TP a pour but de développer une application de réalité augmentée (AR) sous Unity permettant de détecter un objet réel ou une image, d’afficher du contenu virtuel en AR, et d’intégrer une fonctionnalité d’intelligence artificielle.

4.4.2 Technologies utilisées

L’application est développée avec **Vuforia Engine** selon la stratégie de détection par Image Target. Cette approche permet de scanner une image cible (logo, affiche...) et de déclencher

l'affichage d'un contenu 3D et d'une réponse IA associée.

4.4.3 Structure du projet

Le projet est organisé dans le panneau Assets avec les dossiers **Editor**, **Resources**, **Scenes**, **Settings**, **StreamingAssets**, **TextMesh Pro** et **TutorialInfo**. Les packages **XR** et **InputSystem** sont également intégrés, ainsi que l'asset **Vuforia**.

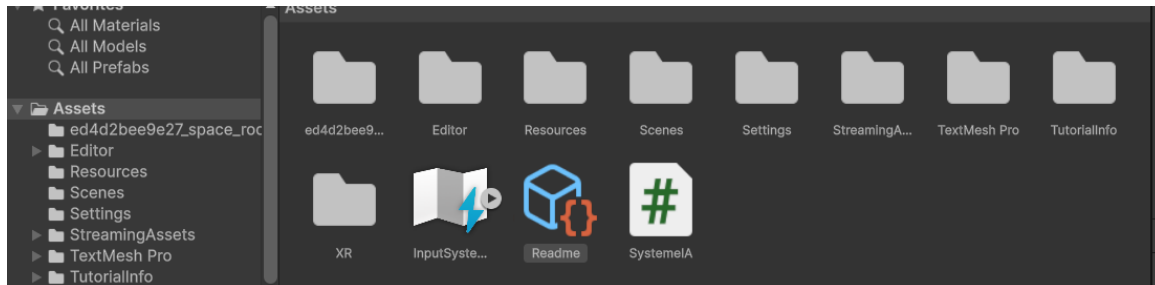


FIGURE 4.5 – Structure du projet AR dans Unity — panneau Assets avec packages Vuforia et XR

4.4.4 Fonctionnement de l'application

L'interface principale présente un bouton **SCANNER** centré à l'écran. Lorsque l'utilisateur appuie sur ce bouton, la caméra s'active et analyse l'environnement pour détecter une image cible configurée dans Vuforia.

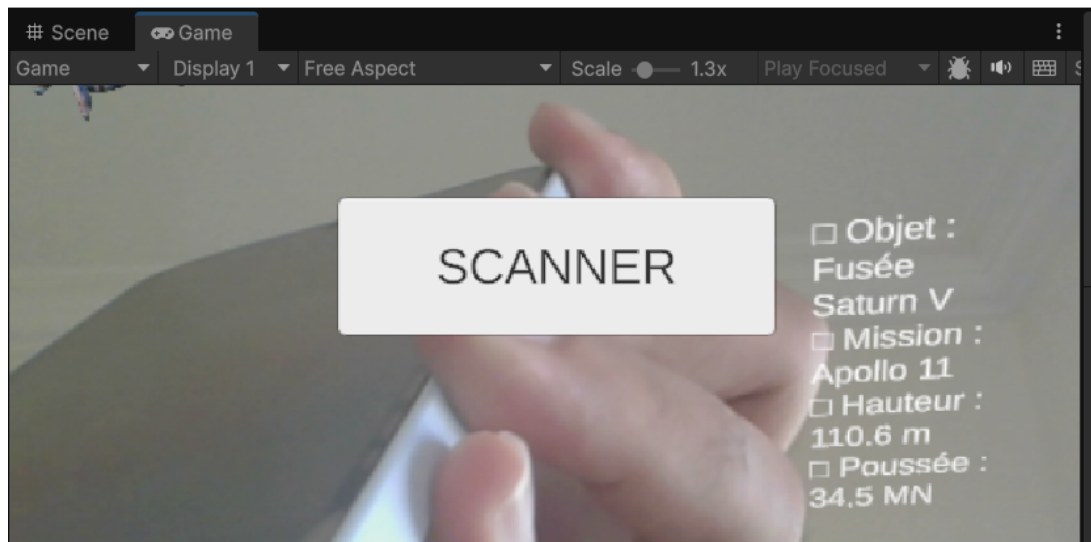


FIGURE 4.6 – Interface AR — bouton SCANNER et affichage des informations sur la fusée Saturn V

Lorsqu'une image cible est détectée (ici, le logo NASA), un modèle 3D de fusée apparaît en réalité augmentée au-dessus de l'image. Simultanément, le module IA génère et affiche des informations contextuelles sur l'objet reconnu. Dans l'exemple présenté, l'IA identifie la fusée Saturn V et affiche ses caractéristiques : mission Apollo 11, hauteur 110,6 m, poussée 34,5 MN.

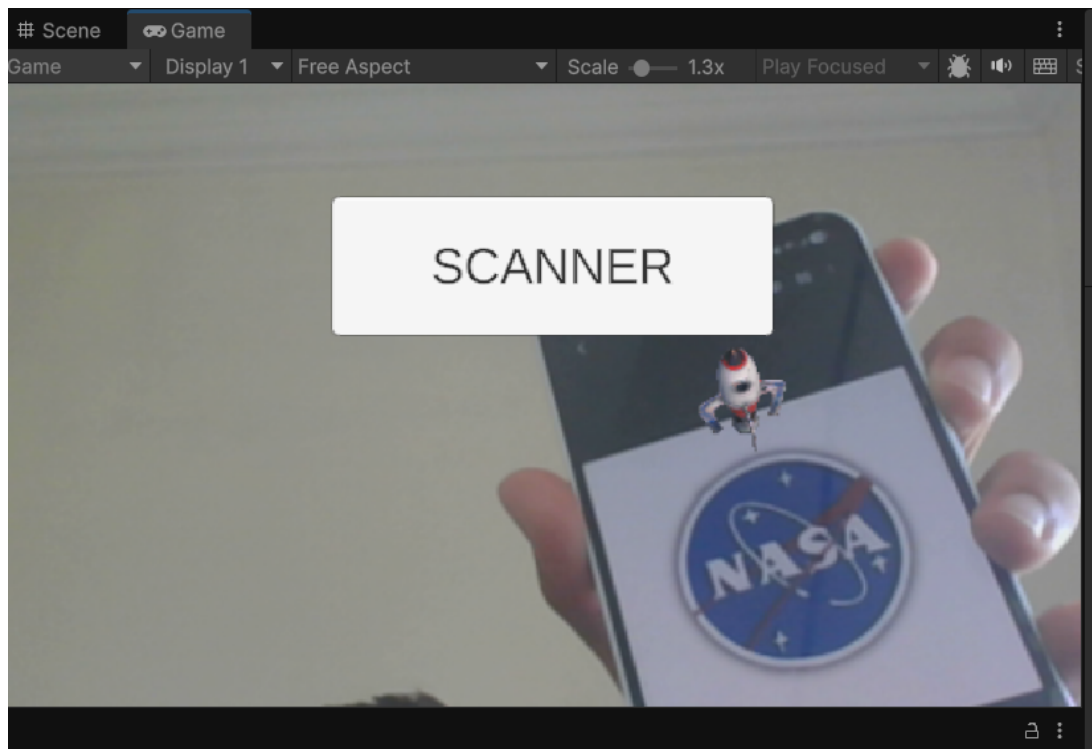


FIGURE 4.7 – Détection AR en action — modèle 3D de fusée affiché sur le logo NASA détecté

En dehors de toute détection, l'application affiche uniquement le bouton SCANNER sur un fond sombre, indiquant que la caméra est active mais qu'aucune cible n'a encore été reconnue.

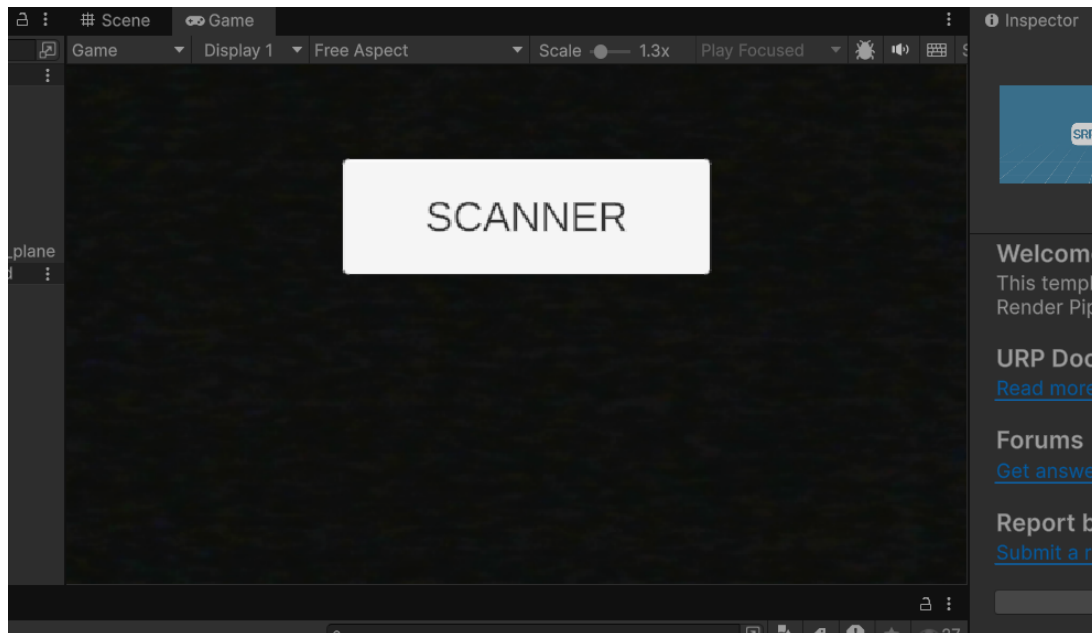


FIGURE 4.8 – État en attente — application AR active, aucune image cible détectée

4.4.5 Pipeline de développement

Le développement suit les étapes recommandées pour Vuforia :

1. Importation du package Vuforia Engine dans Unity.

2. Configuration d'une Image Target avec le logo NASA dans le portail Vuforia Developer.
3. Association d'un modèle 3D de fusée à l'Image Target dans la hiérarchie Unity.
4. Développement du script C# de déclenchement de l'IA lors de la détection de l'image.
5. Affichage dynamique des résultats IA via un composant TextMesh Pro.

Conclusion

L'ensemble de ces travaux pratiques a permis de maîtriser deux outils fondamentaux du génie logiciel moderne : la modélisation UML et la gestion de versions Git. À travers des études de cas progressives et concrètes, nous avons appris à :

- Initialiser et organiser un dépôt Git, gérer des branches et résoudre des conflits de fusion.
- Appliquer le workflow *Git Flow* avec des branches **main**, **develop** et **feature/***.
- Modéliser un système logiciel avec les trois types de diagrammes UML principaux : cas d'utilisation, classes et séquence.
- Relier la conception UML à l'implémentation Git en structurant les branches selon les modules identifiés dans les diagrammes.
- Simuler et résoudre des conflits de fusion dans des situations proches des conditions réelles de travail collaboratif.
- Créer et animer des scènes 3D sous Unity, en appliquant des textures, des Skybox et des scripts C# de mouvement.
- Développer une application de réalité augmentée avec Vuforia, intégrant une détection d'images et un module d'intelligence artificielle.

Ces compétences constituent le socle indispensable pour tout ingénieur logiciel appelé à travailler en équipe sur des projets complexes allant du développement collaboratif à la conception d'expériences immersives.